

Q.write a c program to stimulate first fit,best fit and worst fit

```
#include <stdio.h>
```

```
void firstFit(int blockSize[], int m, int processSize[], int n)
```

```
{
    int allocation[10];

    for(int i = 0; i < n; i++)
        allocation[i] = -1;

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            // allocate only if block is free and big enough
            if(blockSize[j] >= processSize[i] && blockSize[j] != -1)
            {
                allocation[i] = j;

                // mark block as used
                blockSize[j] = -1;

                break;
            }
        }
    }
}
```

```
printf("\n--- First Fit ---\n");
```

```
printf("Process No.\tProcess Size\tBlock No.\n");
```

```
for(int i = 0; i < n; i++)
{
    printf("%d\t\t%d\t\t", i + 1, processSize[i]);

    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}
```

```
void bestFit(int blockSize[], int m, int processSize[], int n)
```

```
{
```

```

int allocation[10];

for(int i = 0; i < n; i++)
    allocation[i] = -1;

for(int i = 0; i < n; i++)
{
    int best = -1;

    for(int j = 0; j < m; j++)
    {
        if(blockSize[j] >= processSize[i] && blockSize[j] != -1)
        {
            if(best == -1 || blockSize[j] < blockSize[best])
                best = j;
        }
    }

    if(best != -1)
    {
        allocation[i] = best;

        // mark block as used
        blockSize[best] = -1;
    }
}

printf("\n--- Best Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for(int i = 0; i < n; i++)
{
    printf("%d\t%d\t", i + 1, processSize[i]);

    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

void worstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[10];

```

```

for(int i = 0; i < n; i++)
    allocation[i] = -1;

for(int i = 0; i < n; i++)
{
    int worst = -1;

    for(int j = 0; j < m; j++)
    {
        if(blockSize[j] >= processSize[i] && blockSize[j] != -1)
        {
            if(worst == -1 || blockSize[j] > blockSize[worst])
                worst = j;
        }
    }

    if(worst != -1)
    {
        allocation[i] = worst;

        // mark block as used
        blockSize[worst] = -1;
    }
}

printf("\n--- Worst Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for(int i = 0; i < n; i++)
{
    printf("%d\t\t%d\t\t", i + 1, processSize[i]);

    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

int main()
{
    int m, n;

```

```

int blockSize[10], processSize[10];
int first[10], best[10], worst[10];

printf("Enter number of memory blocks: ");
scanf("%d", &m);

printf("Enter sizes of %d memory blocks:\n", m);

for(int i = 0; i < m; i++)
{
    scanf("%d", &blockSize[i]);

    first[i] = blockSize[i];
    best[i] = blockSize[i];
    worst[i] = blockSize[i];
}

printf("Enter number of processes: ");
scanf("%d", &n);

printf("Enter sizes of %d processes:\n", n);

for(int i = 0; i < n; i++)
{
    scanf("%d", &processSize[i]);
}

firstFit(first, m, processSize, n);
bestFit(best, m, processSize, n);
worstFit(worst, m, processSize, n);

return 0;
}

```

```
fitc - CodeBlocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Start here X fitc X
117 {
118     printf("%d\t\t%d\t\t", i + 1, processSize[i]);
119 }
120
121
122 Enter number of memory blocks: 5
123 Enter sizes of 5 memory blocks:
124 100 500 200 300 600
125 }
126 Enter number of processes: 4
127 Enter sizes of 4 processes:
128 212 417 112 426
129 {
130
131 --- First Fit ---
132 Process No. Process Size Block No.
133 1 212 2
134 2 417 5
135 3 112 3
136 4 426 Not Allocated
137
138 --- Best Fit ---
139 Process No. Process Size Block No.
140 1 212 4
141 2 417 2
142 3 112 3
143 4 426 5
144
145 --- Worst Fit ---
146 Process No. Process Size Block No.
147 1 212 5
148 2 417 2
149 3 112 4
150 4 426 Not Allocated
151
152 Process returned 0 (0x0) execution time : 20.815 s
153 Press any key to continue.
154
155 firstFit(first, m, processSize, n);
156 bestFit(best, m, processSize, n);
157 worstFit(worst, m, processSize, n);
158
159 return 0;
160
161 }
162
163 }
```

1WA24CS229